zorvyn Backend

Project: Finance data processing & access control

→ build a backend for a finance dashboard where users with diff roles can interact with financial records (income/expenses).
The main focus → clean arch, logical access control (RBAC) and good data modelling.

Core features:

→ Viewer R/O, Analyst (read + insights), Admin (full access).

→ Financial records - so CRUD & filter trans.
[Amount, type, Category, Data, Notes etc].

→ Dashboard Analytics
↳ Endpoints to calc aggregated data.

→ Access Control [OAuth]
↳ Based on Role

⇒ Data Persistence.
DB usage

(*) To be Unique [To Stand Out]
options:

① Maybe JWT [Json Web token]

fast login, Safe, ← | → just like google pass save. and while logging just click to fill., likewise here token

no cons. cheap DB!

② Maybe Pagination

like, google search – 1 Million result but 1Mln result not in single page so as to not crash.

instead

1 page 20 records, next page 20 records and so on.

∴ Fast, won't crash.

Quantity could ask from user, like in bank apps, give statement for 1 week, 10 days, 1 month, 1 Year etc.

can get input from user

③ Rate limiting

∴ No Denial of Service (DoS) attacks.

→ Secure → optimised

↳ even if intentional try to crash website limiting helps prevent it.

1.) Let's use python Along with ① FastAPI ② DJANGO.

DJANGO – good but assessment says it intends to assess how I think

so

FASTAPI ✓.

DB

2) SQLite / PostgreSQL / MongoDB.

✓

Since it's an intern assessment & requirements emphasized simple setup and logical design.

Lets just pick sqlite.

→ File based ∴ No need for Docker or any sw

has ORM (Object Relational Mapper).

So in case, future for production too can be easily modified to postgre.

Since ORM – it's easy, safe, easier code maintenance

Let's also use Pydantic.

↳ Data Integrity, Validation, Type Casting.

∴ Edge Cases handled

instead of "20", "twenty" then crash

3) JWT already mentioned, Cookies.

SQLite ? - Sqlalchemy
engine, SessionLocal - Manages convo. btw python code & SQL.

1/4/26

→ Created Venv.

→ Created database.py

Engine - Actual connection point to database.

Engine - Raw SQL commands.

session - single transaction/unit of work.

SessionLocal - produces sessions

Models.py & database.py - engine, session, sessionlocal

① users. ② Record

users

id	username	hashed_password	role	is_active

Record

id	amount	record_type	category	date	notes	owner_id	is_deleted

is_active, is_deleted, → instead of deleting the whole user just change the values,

So existing and past users data intact.

SECURITY.

① schemas.py ② Auth.py

① → used pydantic schemas to strictly seperate my input/output models from my db models.

Input: passwd creation must on CreateUser et. [USER CREATION]

Output: When frontend asks user data's my passwd doesn't go, ∴ No leak of passwd. [USER RESPONSE]

② usage of bcrypt [passwd hashing func] intentionally slow.

Considerations: MD5 and SHA-256 (Fast Hashes)

	Fast	Slow
purpose:	Data integrity check if file changed.	Passwd security
Speed	Mln's of Hashes per second	10 per second.
Brutforce	Vulnerable, can be guessed	less vulnerable Takes much time to gu
Salting	Manual	Automatic.

Salting: diff hashes.
random string.

Bcrypt generates a random salt for every passwd.
Pswd
Sp looks like: $2b $12 $e9h - - -

But sha ef92 ... same for same char → easy to guess
Didn't use Argon2 as it could be utilised if further improvement
→ (get_password_hash)
Bcrypt - 100 ms delay.
when Brute forcing, for a theif long period of wait.

As Hard/w faster, passwd easier to crack

Work factor ↑ or ↓.
⇓
safety ↑ if WF ↑.
as delay increases.
As theifs use GPU's to generate and guess pass → brute force

This is self-inflicted D.O.S.

Default now: 12 → 12 loop.
as it is considered the goldilocks value.

All these accuses like ability to ↑ WF value is why companies prefer bcrypt//

→ Next
JWT Concept. → (create - access-token)
when user successfully logs in, backend creates a dict. holding their username & role. {"sub": "alice", "role": ...}
A secret key/passcode created → (secret-key) to cryptographically seal that dict. into a string (Token) → given to user
↳ from then on → user gives token
↳ logs in
we instantly know, ~~who role without~~ querying db.

→ How it's done
A JWT → 3 parts.
Header, payload, signature.
JSON
payload { user id: , role: , exp: }
Step A: ↳ encoded to Base 64 URL
not encryption, anyone can decode
Step B:
The server takes 3 things
→ Header (which algo to use)
→ The encoded payload.
→ secret-key (a long, random, known only to server)

Step C. The verification

Secret key
↳ Cannot make out.

can see data, but doesn't know secret_key, so can't get

∴ Never Commit secret_key on github.
∴ Use env file
∴ No hardcode needed.

DEPS.PY → Access Control

Used FASTAPI dependencies to create centralized middleware checkpoint for authentication & authorization.

Who are you? → get_current_user
↳ Every time a user makes a request to a protected endpoint ~~may~~ app forces them to run through this func automatically.
Uses the JWT token un-seals it's secret-key and verifies. Also checks db for if the user is active.

If pass, ~~they~~ it hands user object to the API endpoint.
What do you do) Role Checker.
↳ handles Viewer, Analyst, Admin

Routers/users.py.
The file has 3 endpoints
→ POST/users/register
→ POST/users/token (login)
→ GET/users/me.

① DB dependancy injection
(Depends(get_db)).
→ views of db are presented to every request.
So no crash while many use.
② → OAuth2 PasswordRequestForm for swagger UI compatibility.
No need for Frontend code as Swagger UI gives us the 'authorize' button to test.
③ Return Model Securing
→ strips off sensitive info before ~~giving~~ the return

routers/recordapy
→ security checks [abstracted, ∴ clean]

↳ Before opening db, JSON payload,
run my token checker.
If Admin ✗, 403 error.
↳ soft delete.
No records deleting,
just changing values of
is_deleted.
∴ Data Integrity.
Smart filtering
↳ By inducing dynamic
queries if passed.

(✱) DB aggregation vs application processing

Total Expenses - used SQLAlchemy's
aggregate func. is like func.sum and
group_by
→ why didn't import all records and
write a loop?
If 50k records,
memory crash & optimization - ✗.

So.
db.query(func.sum(models.Record.
amount)).
↳ This is
= Select sum(amount) from
records;
∴ db engine executes internally.

uvicorn - hosts the server.